

Project Overview

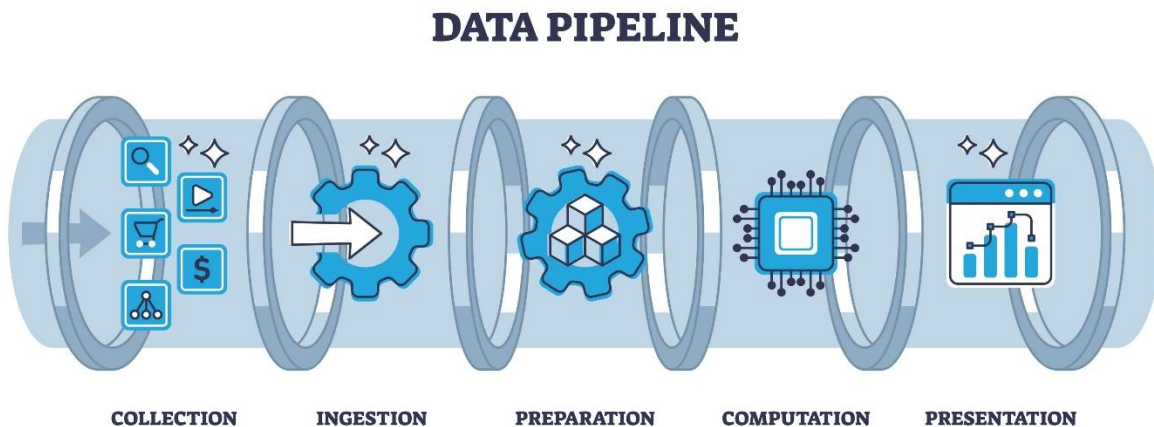
"The aviation sector operates within a highly complex environment where operational efficiency is constantly challenged by unpredictable variables. **AeroPredict** is a comprehensive analytical framework developed to decode the intricate relationship between aviation traffic flow and meteorological dynamics. By synthesizing historical flight logs with granular weather datasets—including temperature, wind speed, and precipitation patterns—our system transitions from reactive scheduling to proactive, data-driven forecasting. This project navigates the full data science lifecycle: from raw data extraction via specialized APIs and rigorous preprocessing to the implementation of a machine learning-based predictive model. The ultimate objective of **AeroPredict** is to provide actionable intelligence that empowers airport operators to mitigate delay risks, optimize resource allocation, and enhance the overall passenger experience through smarter, weather-aware decision-making."

Project Architecture

2.1. System Overview: "The **AeroPredict** architecture is designed as a modular, end-to-end data pipeline. It systematically transforms raw, multi-source inputs—aviation flight logs and meteorological data—into a unified analytical and predictive model. The pipeline ensures data integrity by isolating the ingestion, cleaning, and modeling stages into distinct, maintainable modules."

2.2. The Data Flow Pipeline: "The workflow follows a sequential progression to ensure high-quality training data:

1. **Data Ingestion:** Utilizing specialized API scripts (Aviation_API.ipynb) to fetch historical flight data, coupled with structured weather data processing.
2. **Data Cleaning & Normalization:** Using modular notebooks (Aviation_cleaning, weather_clean) to handle missing values, correct timestamp anomalies, and normalize extreme weather fluctuations via the Yeo-Johnson method.
3. **Relational Integration:** Implementing a time-sync layer that merges flight records with corresponding weather conditions using a unique weather_key (IATA code + Date).
4. **Predictive Analytics:** Passing the synchronized, clean features into a **Random Forest Classifier** to generate delay probability insights."



Data Cleaning & Preprocessing

3.1. Aviation Data Sanitization: "Raw flight logs often contain missing values and inconsistent timestamps. Our approach involved a multi-step cleaning process:

- **Temporal Standardization:** All scheduled and actual flight timestamps were converted to standardized datetime formats to facilitate duration calculations.
- **Delay Anomaly Resolution:** We implemented logical inference scripts to calculate missing delay values based on the difference between scheduled and actual arrivals.
- **Outlier Handling:** To ensure robust model training, we applied **Winsorization** (at the 5th and 95th percentiles) on delay metrics, preventing extreme outliers from skewing the prediction accuracy."

3.2. Meteorological Data Enhancement: "To align meteorological conditions with flight schedules, the weather dataset underwent:

- **Time-Shift Normalization:** A temporal offset was applied to synchronize historical weather records with our current 2026 dataset timeframe.
- **Distribution Transformation:** We utilized the **PowerTransformer (Yeo-Johnson method)** to normalize weather features, effectively reducing data skewness and improving the performance of our machine learning algorithms."

Predictive Modeling & Model Evaluation

4.1. Model Architecture & Selection: "We selected the **Random Forest Classifier** as our primary predictive engine. This choice was driven by its inherent ability to handle complex, non-linear relationships between categorical features (such as IATA codes) and numerical meteorological metrics.

To ensure the model remains robust and generalizes well to unseen flight data, we configured it with `max_depth=8` and enabled `class_weight='balanced'`. This configuration is specifically designed to address class imbalance, ensuring that the model accurately predicts minority delay cases without over-fitting to on-time flights."

4.2. Feature Encoding Strategy: "Handling high-cardinality features—such as airport codes and airline identifiers—required a strategic approach. We employed **OrdinalEncoder** to transform categorical variables into a numerical format suitable for tree-based processing.

By utilizing the `handle_unknown='use_encoded_value'` parameter, our pipeline ensures resilience against unseen categories, allowing the model to make predictions even when encountering new or rare airport codes during live operation."

4.3. Evaluation Metrics: "To quantify model performance, we utilized a 20% test-split strategy. Our evaluation was not limited to accuracy alone; we prioritized the **F1-Score** and **Precision-Recall metrics**.

This ensures that our model strikes a balance between identifying actual delays and minimizing false alarms, providing airport operators with actionable, reliable predictions."

Challenges & Solutions

5.1. Data Temporal Synchronization: "The primary challenge involved a temporal mismatch between flight logs and historical meteorological data. These datasets originated from different time-frequency domains.

- **Challenge:** Aligning flight departure timestamps precisely with weather snapshots at the IATA airport location.
- **Solution:** We developed a custom alignment script that utilized a unique `weather_key` (IATA code + Date). This effectively synchronized the datasets, ensuring that each flight record was mapped to the correct, locally relevant weather conditions, thereby maintaining the integrity of our predictive model."

5.2. Handling Data Imbalance: "In real-world aviation scenarios, flight delays are less frequent than on-time departures.

- **Challenge:** The model was biased towards 'On-time' predictions due to the majority class dominance.
- **Solution:** We implemented the `class_weight='balanced'` parameter within the Random Forest algorithm. This forced the model to assign higher importance to the minority class (delays), significantly improving our recall and ensuring the model remains useful for risk management."

5.3. Pipeline Robustness & Scalability: "Ensuring the GUI could handle unknown or new airport codes was a significant hurdle.

- **Challenge:** Standard encoders fail when encountering categories not present in the training set.
- **Solution:** By configuring the `OrdinalEncoder` with `handle_unknown='use_encoded_value'`, we created a resilient prediction pipeline that handles unseen data gracefully, preventing system crashes and ensuring continuous usability."